SPHERE WEB AUTOMATION

Account Page Script Step-by-Step Walkthrough

A section-by-section map of how `tests/Account.spec.ts` covers the learner's profile page — the bilingual engine behind every assertion, the reason it can never navigate to the page directly, and an honest account of which of its 68 tests can actually fail.

Script tests/Account.spec.ts · 1,239 lines · **68 tests** in 10 groups · 42 expect()

Target /app/profile-view (login required)

Run npm run run:account

Date 12 September 2026

Contents

1	What this script does	Purpose, scope, entry points
2	The bilingual engine	Diagram — one UI, two languages, and a hazard
3	Getting to the page, and staying there	Diagram — one shared tab, and why no direct URL
4	Group 1 — Landing and left nav	13 tests · lines 269–382
5	Group 2 — Personal Details	13 tests · lines 384–581
6	Groups 3 and 4 — Organization and Academic	14 tests · lines 583–704
7	Groups 5 to 8 — the soft middle	12 tests · lines 770–954
8	Group 9 — Logout	4 tests · lines 988–1072
9	Group 10 — Header and page health	12 tests · lines 1074–1239
10	What green actually proves	Diagram — 40 assert · 28 observe
11	Artifacts, run variants, failure modes	Troubleshooting reference

How to read this document

Sections 4 to 9 walk the file in order, one section per `describe` group. Each test is marked by what it actually does with its result:

Marker	Meaning
assert	An `expect()` runs. A missing or wrong element fails the test.
observe	No `expect()` at all. The test performs the action, measures something, and prints it for a human to read.

The split is **40 assert to 28 observe**, and it is not spread evenly: three whole groups contain no assertion at all. Section 10 maps that, and the group sections flag the individual cases as they come up.

> **This spec writes to a real account**
> Most specs in the suite only read. This one fills the phone and pincode fields and saves, sets the organisation name and saves, and **switches the whole UI to Hindi and back**. It is pointed at a shared test learner, and Section 2 covers the safety net that stops a half-finished run leaving that account in Hindi for everything that follows.

> **Line numbers in this document are live**
> Every reference points at `tests/Account.spec.ts` as it stands today — 1,239 lines, including the per-line explanatory comments carried across the suite.

1 What this script does

It is the sweep of the learner's own profile page — the one screen where a user changes something about themselves rather than about a course.

`/app/profile-view` carries five left-nav sections, each with its own form, plus a profile panel, a header, and a logout flow. This spec puts **68 granular tests** across all of it in ten groups, so a failure names the control rather than the page.

The five sections, and what each holds

Section	Fields
Personal Details	First and last name, phone, email (masked), date of birth as three dropdowns, country, pincode, Save
Organization Details	Background, designation, RN/RM number, organisation type, organisation name, Save
Academic Details	An Education block with an `+ Add Education` control
Language Preference	English / हिंदी radios, and Save
My Certificates	The certificate list, or its empty state

Around them: the profile avatar and its pencil edit icon, a **Help and Support** button, a **Logout** button that opens a confirmation modal, and the shared app header.

What makes this page harder to test than it looks

It is bilingual, and it can change

- Every label renders in English *or* Hindi
- One of this file's own tests flips the language
- So English-only selectors break the tests that follow
- Section 2 is entirely about this

It cannot be navigated to

- A direct URL renders the Account route **blank**
- So does a plain `page.reload()`
- Every entry goes Home → header click
- Section 3 covers the machinery

It writes to a shared account

- Phone, pincode and organisation name are saved
- The language is switched and switched back
- Fields are read-only on production
- So the mutating tests guard before they type

Entry points

Command	What it runs
`npm run run:account`	All 68 tests, headed, against staging. The full walkthrough this document describes.
`npm run report:app`	This spec alongside My Courses, Course TOC and Competency — the authenticated-app blob report.
`… --grep "Logout"`	One group. Every group name starts with `Account —`, so a group name is unambiguous.
`… --grep "Language Preference"`	The group most worth running alone, because it is the one that changes shared state.

> **The file header promises independence, and the code mostly delivers**
> "Each test is INDEPENDENT — reuses `.auth-state.json` from previous tests." There is no `describe.serial` anywhere, and a `beforeEach` resets the viewport, dismisses any leftover modal and re-

> opens the Account page if a previous test navigated away. What the tests *do* share is a single browser tab and a single real account — so independence holds for ordering, not for state. Section 3 covers the difference.

2 The bilingual engine

Every text assertion in this file matches English or Hindi. That is not future-proofing — it is a direct consequence of one of its own tests.

The Language Preference group switches the account to Hindi, saves, and checks the choice survives a reload. From that moment the entire UI is in Hindi, including the left-nav labels every later section needs to click. An English-only selector would not fail with a useful message; it would simply find nothing, and the rest of the run would collapse for a reason that had nothing to do with the page being broken.

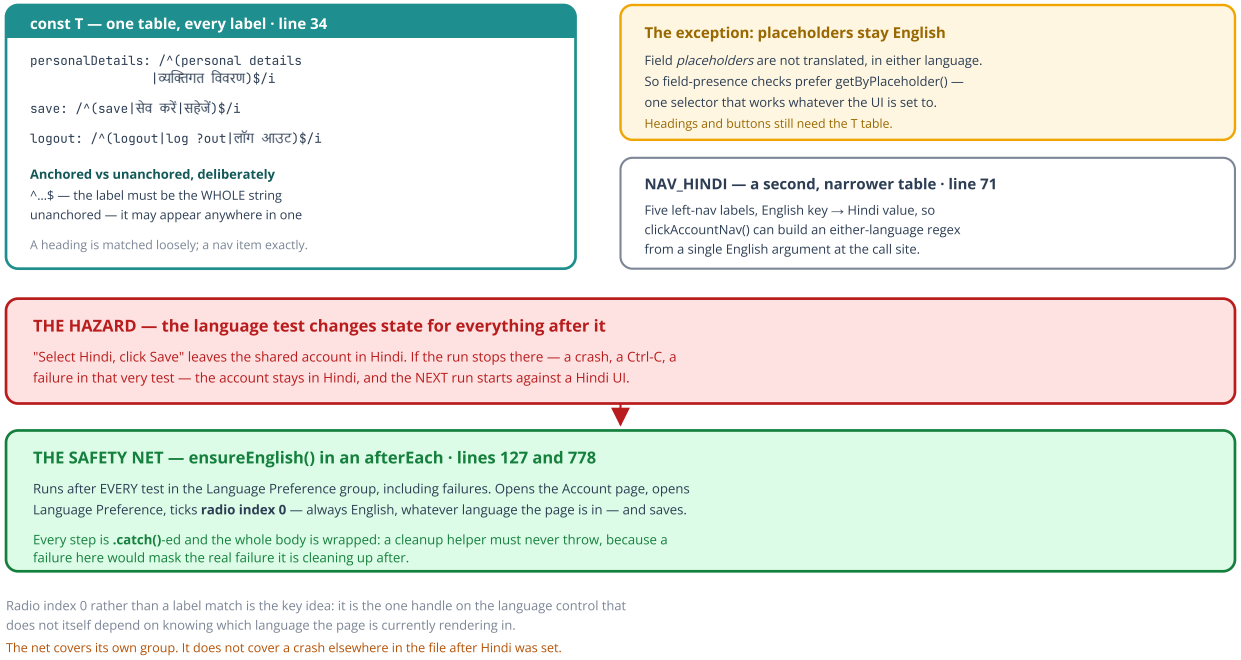


Figure 1 — two lookup tables, one exception, and the cleanup hook that keeps the shared account usable for the next run.

3 Getting to the page, and staying there

The Account route does not render on direct navigation. Everything about this file's setup follows from that one fact.

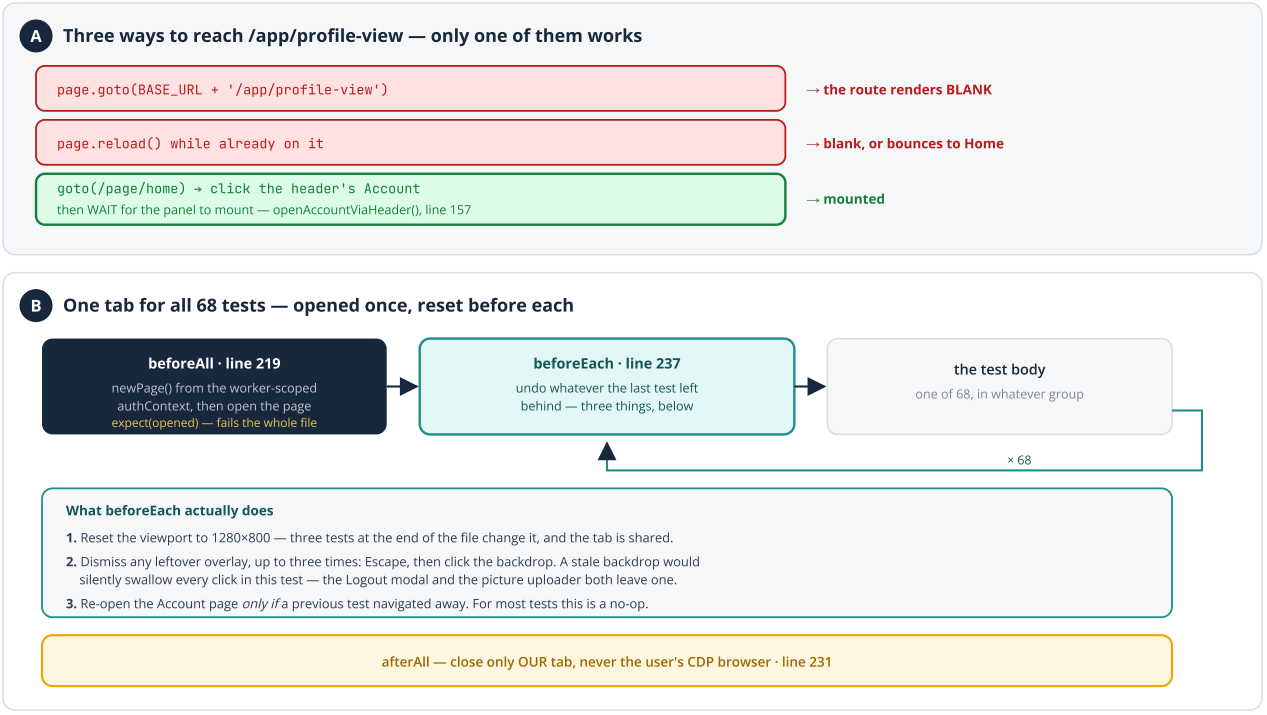


Figure 2 — (A) the only navigation that works, and (B) the shared-tab lifecycle built around it.

Why `openAccountViaHeader()` retries

Three attempts, and each one re-does the whole hop rather than retrying just the click. Two details are worth calling out:

Choice	Reason
`domcontentloaded`, never `networkidle`	The comment is emphatic and correct: **this SPA never goes idle** — long-polling and analytics keep the network busy, so `networkidle` would simply time out on a perfectly healthy page.
Clicking is not enough	After the click it waits for `accountMountedMarker()` — the Position label or the Personal Details nav item, whichever appears — with an 8-second window. A click that navigates but leaves the panel unmounted counts as a failed attempt.

The header control itself is matched three ways (line 98): `button`, then `link`, then plain text. The source explains why — **it renders as a `<button>` on this SPA, not a link**, and it sometimes lags behind first paint, which is also why it gets a generous 12-second visibility window.

> **One helper name carries the whole lesson**
> `reNavigateToAccount()` (line 191) is a one-line wrapper around `openAccountViaHeader()`, and it exists purely so that reload-then-verify tests read correctly. Its comment says: *"Use this instead of `page.reload()`, which makes the Account route render blank."* Naming the wrapper after the intention — re-navigate — rather than the mechanism is what stops the next person reaching for `reload()` and spending an afternoon on a blank page.

> **Shared tab, shared account, independent order**

> Because there is one tab and one real learner, tests are not isolated from each other's *effects* — the language test changes what later tests see, and the save tests change stored data. What `beforeEach` buys is that the **order** does not matter and one failure does not cascade: each test starts on a mounted Account page at a known viewport with no modal open, whatever the last one did.

4 Group 1 — Landing and left nav

Thirteen tests, lines 269–382. The profile panel and the five nav items — the fixed chrome that every later group depends on.

#	Test	Locator	What it proves
1	**URL contains /profile** line 271	`expect(page.url()).toMatch(/profile|account/i)`	assert We are where we think we are. Accepts either route spelling, because the app has used both.
2	**User name in left panel** line 278	`locator('h1, h2, h3, [class*="user-name"]') .filter({ hasText: /\w+\s+\w+/ }).first()`	observe Reads whatever matches a crude "two words" shape and prints it. A blank name passes, because the name is account data rather than page structure.
3	**Position label** line 290	`expect(accountMountedMarker(page)).toBeVisible()`	assert The label's *presence* is required; its content is only logged. This is the same marker `openAccountViaHeader()` waits on, so it doubles as a mount check.
4	**Profile avatar** line 300	`'[class*="avatar"] img, [class*="profile-pic"] img, [class*="user-image"]'`	observe Three class conventions, one printed boolean. Correctly soft: a learner with no photo still has a valid account.
5	**Pencil edit icon** line 309	`button:has(mat-icon:has-text("edit")) mat-icon:has-text("edit") [aria-label*="edit" i] · [class*="edit-icon"]`	observe Four shapes because Material icons can be found by ligature text, by label, or by class depending on build.
6–10	**The five nav links** lines 323–348	`expect(getByText(T.personalDetails).first()) … and four more, one per section`	assert All five hard, and rightly so: the nav is fixed chrome, identical for every account, so it is safely assertable in either language.
11	**Help and Support button** line 354	`getByRole('button', { name: T.helpSupport }) .or(getByText(T.helpSupport))`	assert Button first, text as a fallback.
12	**Logout button** line 362	`getByRole('button', { name: T.logout }) .or(getByText(T.logout))`	assert Same shape. Group 9 then exercises what it opens.
13	**Personal Details active by default** line 370	`read the class attribute, print it`	observe The class is printed rather than checked for an active marker. See the note below.

Four "active in nav" tests, none of them assertions

Test 13 here, plus the equivalents in the Academic Details and My Certificates groups and the header's *"Account nav link is marked active"*, all follow the same shape: read `class`, print the first hundred characters, pass. None of them looks for an active/selected marker in that string.

That is defensible while the active-state class name is in flux — asserting on `mat-list-item-active` would break on a Material upgrade that renames it. But four tests named "is active" that cannot detect an inactive item is worth knowing when reading a green report. A single shared helper checking for `/active|selected/i` would turn all four into real checks without hard-coding one build's class name.

The group's shape is otherwise sound: **9 of 13 assert**, and the four that observe are all things that legitimately vary by account — the name, the photo, the edit affordance, and a class string.

5 Group 2 — Personal Details

Thirteen tests, lines 384–581. The largest form on the page, and the first group that writes to the account.

#	Test	Approach	What it proves
1–4	**Heading, first name, last name, phone** lines 391–418	`getByPlaceholder(/first name/i)` … `getByPlaceholder(/phone/i)`	assert All four hard. Placeholders are used rather than labels — **the one thing on this page that is not translated**, so one selector works in both languages.
5	**Email field is visible and masked** line 423	`expect(getByPlaceholder(/email/i)).toBeVisible()` then walk every non-radio input for a value with `@` `masked = /\*/.test(value)`	assert on presence, observe on masking. The test name promises both; only the first half is enforced. An unmasked email would print `masked: false` and pass.
6–9	**DOB block, country, pincode, Save** lines 449–473	`getByText(/date of birth|जन्म की तारीख/i)` … then `saveButton(page)`	assert Four more hard checks. The DOB test also counts the native `<select>` elements and prints the total, which is what tests 12 and 13 later index into.
10	**Fill Phone and Pincode then Save** line 481	`isEditable()` guard on each field `isEnabled()` guard on Save read the toast, print it	observe The most carefully written test in the group, and still it asserts nothing. See below.
11	**Saved values persist after reload** line 528	`reNavigateToAccount()` → re-read both inputs → print	observe **Nothing compares the re-read values to what test 10 typed.** See the red note.
12–13	**DOB Year / Country can be changed** lines 546, 563	`selectOption({ index: 5 }) / ({ index: 2 })` read the value back, print it	observe Positional: Year is select 0, Country is select 3. Both guard on the count first, and both swallow the selection error, so a dropdown that refuses to change still passes.

The guards on test 10 are the right instinct

On production the phone field is read-only — it is verified and masked — so a blind `fill()` would throw for a reason that is not a defect. The test checks `isEditable()` first, and does the same for pincode. Then it checks `isEnabled()` on Save, because **the button stays disabled until an editable field actually changes**: if nothing was editable, a disabled Save is correct behaviour, not a bug.

And when it cannot save, it says so honestly rather than pretending:

```
ℹ Save button is disabled (no editable fields changed) — nothing to save
```

Test 11 is named for the one thing it does not do

"Saved values persist after page reload" re-navigates, re-reads the phone and pincode fields, and prints them. It never compares them to `'9876543210'` and `'560001'` — the values test 10 typed. A save that silently failed, or a backend that dropped the write, produces exactly the same green result.

The fix is two lines and needs no new machinery: have test 10 record what it actually filled (it already knows, because it guards on `isEditable()`), and have test 11 assert against that. As written, the two most valuable tests in the group — write it, then prove it stuck — prove nothing between them. The identical problem exists in Organization Details, Section 6.

6 Groups 3 and 4 — Organization and Academic

Fourteen tests, lines 583–765. The weakest stretch of the file: three assertions across ten Organization tests, and the same write-then-don't-check pattern as Section 5.

Organization Details 10 tests · 3 assert · 7 observe

#	Test	Approach	What it proves
1–2	**Heading, Background label** lines 590, 598	`filter({ hasText: T.organizationalHead }) getByText(T.background)`	assert Both hard, in either language. Note the heading and nav labels differ — *"Organizational Details"* as a heading, *"Organization Details"* in the nav — which is why `T` carries two separate entries for it.
3	**Background shows a value** line 606	`getByText(/healthcare worker|volunteer|asha |student|faculty|others/i)`	observe Any of the six known options, printed. Soft is right here — which background this learner chose is account data.
4	**Designation dropdown is visible** line 618	`locator('select, mat-select, [role="combobox"]') .count()`	observe **It counts every dropdown on the page and prints the total.** Nothing ties any of them to a designation. A page with the designation control missing, but a country dropdown present, passes.
5–7	**RN/RM, org type, org name** lines 629, 638, 647	`isVisible().catch(() ⇒ false), printed`	observe Three tests, three booleans, no assertions. Each would read identically on a page where the field is simply absent.
8	**Save button visible** line 656	`expect(saveButton(page)).toBe Visible()`	assert The third and last hard check in the group.
9	**Fill Organization Name and Save** line 662	`if (visible) { fill('Test Hospital Bangalore'); click Save; print the toast }`	observe Writes real data. Two differences from its Personal Details sibling: **no `isEnabled()` guard on Save**, and the whole body sits inside a visibility guard — so a missing field is a silent pass, not even a logged skip.
10	**Saved name persists after reload** line 687	`reNavigateToAccount() → re-read → print`	observe The same gap as Section 5: the value is read back and printed, never compared to `'Test Hospital Bangalore'`.

The one test here that would catch a real regression is one line away

Test 10 already re-navigates, already re-opens the section, and already reads the field. Adding `expect(val).toBe('Test Hospital Bangalore')` would turn the pair into the only end-to-end write-and-verify in the whole spec — and it is the sort of thing that breaks when a profile API changes shape, which is exactly what you want a test to notice.

The same single line applies to Personal Details test 11. Two lines across the file would close both.

Academic Details 4 tests · 2 assert · 2 observe

#	Test	Approach	What it proves
1	**"Education" heading** line 713	`getByText(T.education)`	assert Unanchored, because the heading can carry extra text around the word.

#	Test	Approach	What it proves
2	**"+ Add Education" link** line 722	`getByText(T.addEducation)`	assert The `T` entry begins `/(\+?\s*add education…` — the leading optional plus is there because the control renders as "+ Add Education" on some builds and "Add Education" on others.
3	**Clicking it opens a form or modal** line 729	`click, then look for a dialog, new inputs, or education-ish wording; print what was found`	observe The click is real; the outcome is printed. A control that opens nothing at all would pass.
4	**Academic Details active in nav** line 757	`read the class, print it`	observe The second of the four "is active" tests from Section 4's note.

Both groups open their section through a nested `beforeEach` that calls `clickAccountNav()` — so every test starts on the right panel without repeating the navigation, and the either-language regex is built once from the English name at the call site.

7 Groups 5 to 8 — the soft middle

Twelve tests, lines 770–954. Language Preference is the only one of the four with real teeth; the other three contain no assertion at all.

Language Preference 7 tests · 4 assert · 3 observe

#	Test	Approach	What it proves
1–3	**Heading, English radio, Hindi radio** lines 783, 792, 800	`getByText(/^(english|अंग्रेजी)$/i)` `getByText(/^(hindi\|हिंदी)$/i)`	assert All three hard. Note that "English" itself renders as **अंग्रेजी** when the UI is in Hindi — the option labels are translated along with everything else, which is precisely why `ensureEnglish()` uses the radio's *index* instead.
4	**English selected by default** line 807	`read which radio is checked, print it`	observe Defensible: the account's language is state this very group changes, so a hard assertion would depend on run order.
5	**Save button visible** line 828	`expect(saveButton(page)).toBeVisible()`	assert
6	**Select Hindi, save, verify after reload** line 834	`check the Hindi radio, Save, re-navigate, read which radio is checked, print it`	observe **The test that changes shared state for the whole file**, and it does not assert that the change stuck. Its `afterEach` then undoes it.
7	**Switch back to English, verify** line 862	`the mirror image of test 6`	observe Same shape, same gap. Between them these two are the strongest evidence the page's language setting works — and neither can fail.

My Certificates, Profile Picture, Help and Support 5 tests · 0 assert

Group	Test	Approach	What it proves
My Certificates line 887	Active in left nav · 894	`read the class, print it`	observe The third "is active" test.
	"No Certificates" empty state · 901	isVisible(), printed	observe `Correctly soft — this account *may* have certificates, so requiring the empty state would be wrong.`
	No certificate cards · 910	count(), printed as `N (expected 0 for new user)`	observe `The parenthetical states an expectation the code does not enforce — the clearest example in the file of a test *describing* a check rather than performing one.`
Profile Picture line 923	Pencil opens an uploader · 925	`click, then count input[type="file"] and look for upload wording`	observe Returns early with a log line if the pencil is absent. A pencil that opens nothing prints `file inputs: 0` and passes.

Group	Test	Approach	What it proves
Help and Support line 956	Clicking it navigates or opens · 958	`click, then print the URL and whether any helper UI appeared`	observe Same shape. The test name's "navigates / opens helper" is an accurate description of what it *watches for*, and of what it does not require.

Three of these five are soft for good reasons; two are not

The certificate empty-state and the background-value tests are correctly soft: they depend on what this particular account holds, and hardening them would make the suite fail on a different learner. But **the pencil and Help and Support tests are about page structure, not account data** — the controls either open something or they do not, for every account alike. Those two are the ones worth hardening, and each needs one `expect()` on the signal it already measures.

8 Group 9 — Logout

Four tests, lines 988–1072, and the only group in the file where every single test asserts. It is also the group with the clearest reason to.

Logout is destructive and irreversible within a session. A Logout button that signs the user out *without asking* is a real defect, and so is a confirmation dialog that offers only one way out. Both are things a test can state precisely, and this group does.

#	Test	The assertion	What it proves
1	**Logout opens a confirmation** line 990	`expect(getByText(/logout now|लॉग आउट|साइन आउट/i).or(getByRole('button', { name: /^(yes|हाँ|हां)$/i })).first()).toBeVisible({ timeout: 5_000 })`	assert **Real coverage.** An immediate logout with no confirmation fails this outright. Either the heading or the Yes button is enough proof the modal opened.
2	**Modal asks "Are you sure?"** line 1006	`expect(getByText(/are you sure\??|क्या आप सुनिश्चित|क्या आप वाकई/i).or(getByRole('button', { name: /^(no|नहीं)$/i }))`	assert Two Hindi phrasings, because the app has used both. The No button is accepted as a fallback for the same reason as test 1.
3	**Both Yes and No are offered** line 1020	`expect(yesBtn).toBeVisible()` `expect(noBtn).toBeVisible()`	assert Two separate assertions. The source's comment names the defect it guards against exactly: *"a modal with only Yes would be a trap."*
4	**"No" cancels and keeps the session** line 1038	`expect(page.url()).toMatch(/profile|account/i)` `modal-closed and still-signed-in: printed only`	mixed The URL check is hard — cancelling must not navigate away. The other two signals it gathers are logged, not asserted. See the note.

Every test re-opens the modal

Tests 2, 3 and 4 each begin by clicking Logout again, because the file's `beforeEach` dismisses any leftover overlay — so each test starts from a clean page and has to reopen what it wants to inspect. That is four clicks on a Logout button in one run, and it is why the overlay-dismissal loop in `beforeEach` matters: without it, the second test's click would land on the first test's backdrop.

Test 4 measures the two things that matter most, and asserts neither

After clicking No it gathers three signals: the URL (asserted), whether the modal is gone, and whether the profile marker is still visible. The last two are printed:

```
✅ Modal closed (logout now text hidden): true
✅ User still authenticated (profile marker visible): true
```

The source is candid about the consequence — *"a No that silently logged the user out would still be reported as a pass"* — because the URL would remain on the profile route for a moment even after the session was dropped. Two `expect()` calls on values the test already computes would make this the strongest test in the file rather than the most nearly-strong one.

Taken together the group is still the best-shaped in the spec: **4 of 4 assert**, the assertions are about behaviour rather than presence, and each names a genuine failure a user would notice. The gap in test 4 is narrow and one line wide.

9 Group 10 — Header and page health

Twelve tests, lines 1074–1239. The shared app chrome, three viewports, and the file's only test that opens a second browser context.

#	Test	Approach	What it proves
1–2	**Logo, search bar** lines 1076, 1085	`expect(…).toBeVisible()`	assert Shared header chrome, identical on every page — safely hard.
3	**"Account" nav marked active** line 1096	`read the class, print it`	observe The fourth and last "is active" test.
4	**Clicking Home navigates** line 1104	`click Home, then assert the URL`	assert A behaviour check, not a presence check. Note this test deliberately navigates away — `beforeEach` puts the next one back.
5–6	**Dark-mode toggle, notification bell** lines 1119, 1131	`isVisible(), printed`	observe Both are optional chrome that has come and gone across builds, so soft is defensible here.
7	**No console errors on load** line 1138	`attach a console listener, wait, count, print`	observe **Two problems, not one.** See the red note.
8–10	**Mobile / tablet / desktop** lines 1158, 1168, 1176	`setViewportSize(…)` `expect(accountMountedMarker(page)).toBeVisible()`	assert Three hard checks at 390×844, 768×1024 and 1440×900. The profile panel must survive each layout. `beforeEach` resets the viewport afterwards, which is why it does that at all.
11	**URL persists on reload** line 1184	`a genuine page.reload()` `expect(url).toMatch(/profile|account|home/i)`	assert Deliberately permissive: on production, reloading the profile route bounces to Home, and both outcomes are well-defined correct behaviour. The test then logs whether content actually rendered rather than failing on the known-blank case.
12	**Unauthenticated access** line 1210	`a fresh browser.newContext() with no cookies` `goto the profile URL, then expect(authedVisible).toBeFalsy()`	assert **The most valuable test in the group.** A brand-new context has no session, so the profile must not render — it must bounce to login or public home. This is the one security-shaped check in the file.

Test 7 cannot see the errors it is named for

"No console errors on Account page load" attaches its `page.on('console')` listener **after** the page has already loaded — `beforeEach` navigated to Account before the test body ran. The listener can therefore only catch errors that fire from that moment on. Load-time errors, which are the ones the title is about, have already happened and are gone.

The source is honest about it in a `NOTE` comment, which is better than hiding it, but the test name is not. And it has no `expect()` either: it counts, prints the first five truncated, and passes. So it is soft about a thing it cannot observe. The fix is to attach the listener and *then* navigate — `reNavigateToAccount()` already exists for exactly this — and to assert on the count, or at least to rename the test to say it samples post-load errors.

Test 12 is wrapped, and for a good reason

Creating an isolated context is the whole point of the unauthenticated test, but **a CDP-attached browser cannot always create one** — you are driving someone's real Chrome, not a fresh instance. The test wraps

the creation and calls `test.skip()` with a message naming that reason — skipping honestly rather than passing for the wrong reason. In CI, where `storageState` mode gives it a proper isolated browser, the real check runs.

10 What green actually proves

Forty of the sixty-eight tests assert. The twenty-eight that do not are concentrated, not scattered — and that is the useful part.

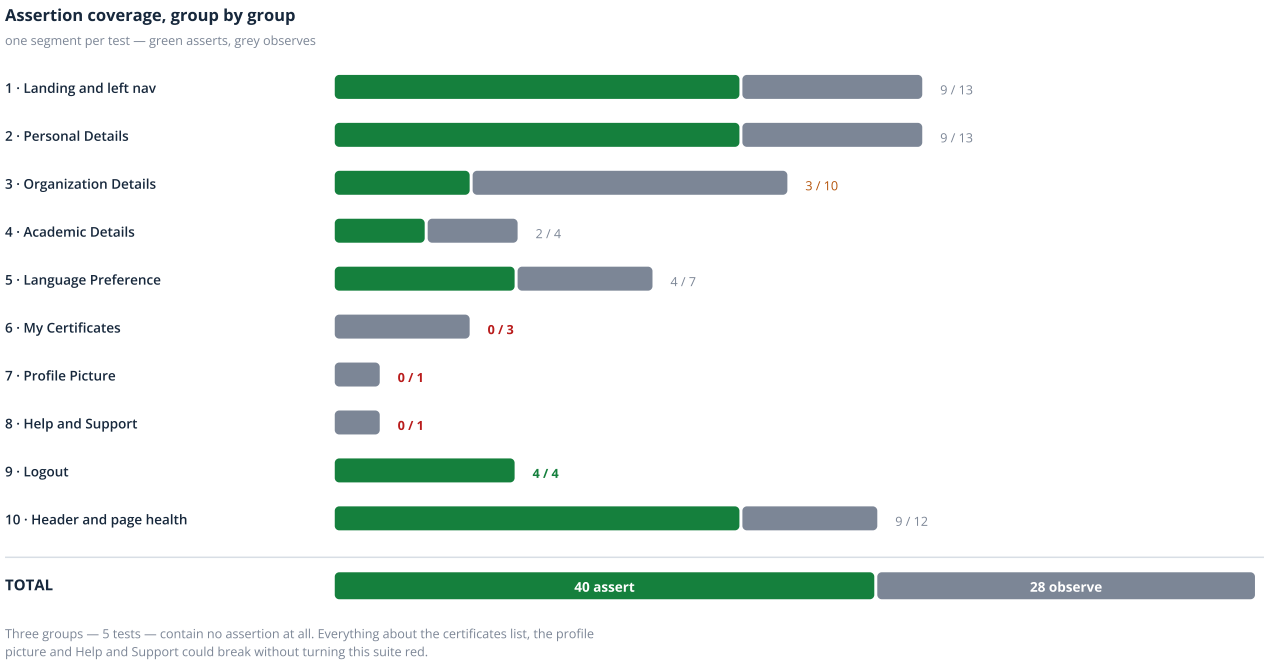


Figure 3 — where the assertions are. The two ends of the file are strong; the middle is where a regression would go unnoticed.

Green means this, for certain

- The Account page mounts, reached the only way that works
- All five left-nav items render, in either language
- Personal Details' fields and Save button are present
- Both Save buttons render
- Language options render and are labelled
- **Logout asks first, and offers both Yes and No**
- Cancelling does not navigate away
- The page survives three viewports
- **A logged-out visitor cannot see the profile**

Green does *not* mean this

- That anything saved actually persisted — both persistence tests only print
- That the email is masked
- That a designation dropdown exists — only that *a* dropdown does
- That any nav item is highlighted when selected
- That the pencil opens an uploader
- That Help and Support does anything
- That the certificates list is correct
- That the page loads without console errors
- That cancelling logout keeps the session alive

The one change that would do the most

Of the eleven gaps above, four share a shape: the test already computes the value it needs and then prints it instead of asserting on it. Those are the cheap ones, and together they are four lines:

Test	The line to add	What it would catch
Personal Details 11 · line 528	`expect(pincodeVal).toBe('560001')`	A profile save that silently fails.
Organization Details 10 · line 687	`expect(val).toBe('Test Hospital Bangalore')`	The same, on the other form.
Logout 4 · line 1038	`expect(userStillVisible).toBe(true)`	A No button that logs the user out anyway.

Test	The line to add	What it would catch
Personal Details 5 · line 423	`expect(masked).toBe(true)`	An unmasked email address on screen.

None of them needs a new locator, a new helper or a new wait — every value is already in a variable by the time it is logged.

Against the rest of the suite

Suite	Tests	Assert always	Shape of the rest
Course TOC	21	15 · 71%	3 conditional, 3 observe.
Public Page	64	42 · 66%	22 soft or present-only.
Public Login	29	18 · 62%	No test asserts that login succeeded.
Account this one	68	**40 · 59%**	28 observe, concentrated in three assertion-free groups and the two save-persistence pairs.
My Courses	49	17 · 35%	31 observe — content depends on what the account studied.

Account sits in the middle, and its softness has a different cause from My Courses'. There, the page genuinely shows whatever the learner happens to have studied, so soft checks are the honest choice. Here, **most of what is left soft is page structure** — a pencil icon, a support button, a nav highlight — which is the same for every account and could be asserted outright.

11 Artifacts, run variants, failure modes

What a run leaves behind

The standard reporters, with nothing bespoke: the branded HTML report in `brand-reports/`, the full run report (HTML and PDF) in `test-reports/run-reports/`, an Excel summary, and a screenshot of **every** test — `screenshot: 'on'`, not just on failure — so a green run still carries visual evidence of all 68.

Run variants

Command	Use it when
`npm run run:account`	Normal use. Headed, against staging.
`HEADLESS=true npm run run:account`	Unattended. Screenshots are still captured.
`… --grep "Logout"`	The group worth running alone when touching the modal.
`… --grep "Language Preference"`	The group that changes shared state. Running it alone still leaves the account in English, because its `afterEach` runs either way.
`USE_CDP=1 …`	Local development against a Chrome you logged into by hand. Note the unauthenticated test may skip in this mode.

Failure modes

Symptom	Cause	Fix
Every test fails at once, with "Account page failed to mount"	`beforeAll`'s assertion. The header's Account control never appeared, across three attempts.	The message names the likely cause: is `TEST_USER` logged in on the environment `BASE_URL` points at? Check both together — a valid session on the wrong environment looks identical.
A run goes red **from the Language Preference group onwards**	The account is in Hindi and something is matching English only.	Any new assertion must go through `T` or `NAV_HINDI`. If the run was interrupted mid-group, `ensureEnglish()` may not have fired — run that group alone once to reset the account.
Clicks land on nothing, several tests in a row	A leftover modal backdrop. The Logout modal and the picture uploader both leave one.	The `beforeEach` loop should have cleared it. If a new overlay shape has appeared, add it to the three-selector list at line 247.
The Account panel renders blank	Something reached the route by URL or by `reload()`.	Use `reNavigateToAccount()`. This is the single most common mistake when adding a test to this file, and the reason that wrapper exists.
`networkidle` timeouts	This SPA never goes idle — long-polling and analytics keep the connection busy.	Wait on `domcontentloaded` plus a specific element, the way `openAccountViaHeader()` does.
The unauthenticated test skips	A CDP-attached browser cannot create an isolated context.	Not a defect — the skip message says so. The check runs properly in CI under `storageState` mode.
A save test passes but the data did not change	The field was read-only, or the save failed silently.	Read the log: `ℹ Phone field is read-only — skipping fill` and `ℹ Save button is disabled` both mean nothing was written. This is exactly the case the missing assertions from Section 10 would surface.

Symptom	Cause	Fix
Viewport tests fail but nothing else does	The profile panel does not survive a narrow layout.	A genuine responsive defect, and one of the few things this file asserts hard at all three sizes.

Where to start when a run goes red

Check the failure count first. **One** failure is usually a real finding in that control. **All 68** is the `beforeAll` mount assertion and means a session or environment problem, not a page problem. **Everything from test 40 onwards** points at the language switch.